LICHEN Technical Assessment Report

LICHEN - Technical Assessment Report

Candidate: Siddheshwar **Role:** Data Scientist, IndiskaAI **Repository Reviewed:** https://github.com/oxpig/LICHEN **Date:** June 2026

---

1. Introduction

This report covers my review of LICHEN, an open-source tool from the Oxford Protein Informatics Group (OPIG) that generates antibody light-chain sequences conditioned on a given heavy-chain sequence. The goal of this assessment was to set up the project locally, understand how it works end to end, review the code, and suggest improvements that would make it easier to use and maintain. I have already completed the local setup and confirmed the model runs correctly, so this report focuses on the architecture review, identified issues, and improvement recommendations.

2. High-Level Overview

LICHEN stands for "Light-chain Immunoglobulin sequence generation Conditioned on the Heavy chain and Experimental Needs." At its core it is a sequence-to-sequence Transformer model (an encoder-decoder, similar in spirit to the original "Attention is All You Need" architecture) that has been trained on paired heavy and light antibody chains.

Given a heavy chain amino acid sequence, the model can:

- Generate one or more compatible light chain sequences.
- Accept hints about what kind of light chain is wanted, such as kappa vs lambda, a specific V-gene family (e.g. IGKV1), or even a specific gene (e.g. IGKV1-39).
- Accept a custom "seed" sequence to start the light chain from.
- Graft specific CDR (complementarity-determining region) sequences into the generated light chain at the correct positions, using either IMGT or Kabat numbering conventions.
- Generate a single light chain that pairs with two different heavy chains at once, which is useful for bispecific antibody design.
- Score how likely a given heavy/light pairing is, via log-likelihood and perplexity.

It can also optionally filter the generated sequences using three external tools: ANARCII (checks that the sequence can be numbered correctly and that CDRs were grafted in the right place), Humatch (predicts how "human" the sequence looks, which matters for therapeutic antibodies), and AbLang2 (scores sequence confidence and can be used to pick the most diverse set of outputs).

The project can be used either as a Python library (`from lichen import LICHEN`) or from the command line via the `lichen` command, and there is also a hosted web tool for people who don't want to run it locally.

3. Architecture and Workflow

The overall flow, from input to output, looks like this:

1. The user provides a heavy chain sequence (and optionally constraints like germline seed, CDRs, or a custom seed).
2. The pretrained model weights are loaded once when the `LICHEN` object is created.
3. The heavy chain is tokenized — each amino acid is mapped to an integer token using the vocabulary in `vocab.json`.
4. The encoder processes the tokenized heavy chain and produces a "memory" representation.
5. The decoder generates the light chain one token at a time. At each step it either: uses a forced token (if a seed or CDR graft applies at that position), or samples from the model's predicted distribution using nucleus (top-p) sampling with a temperature parameter.
6. Once generation is complete (or the end token is produced), the tokens are converted back into an amino acid sequence.
7. If filtering was requested, the candidate sequences are passed through ANARCII, Humatch, and/or AbLang2, duplicates are removed, and the best/most diverse sequences are selected.
8. The final sequences (and optionally their scores) are returned to the user, either as a Python list/DataFrame or written to a CSV file from the CLI.

This is a fairly classic "pipeline" architecture: input handling, preprocessing, model inference, postprocessing/filtering, and output formatting are each handled by separate, fairly self-contained modules.

4. Key Components and Responsibilities

The source code lives under `src/lichen/`. The main pieces are:

`pretrained.py` is the main entry point for anyone using LICHEN as a library. It defines the `LICHEN` class, which loads the model once and exposes the high-level functions: `light_generation`, `light_generation_bulk`, `light_log_likelihood`, and `light_perplexity`. This is also where input validation and the filtering logic are orchestrated.

`model.py` contains the actual PyTorch model definition: positional encoding, token embeddings, and the `Seq2SeqTransformer` class (6 encoder layers, 6 decoder layers, 512-dimensional embeddings, 8 attention heads).

`load_model.py` handles loading the model weights from disk, picking the right device (CPU or GPU), and figuring out how many CPU threads to use if running on CPU.

inference.py contains the `Heavy2Light` class, which implements the actual decoding loop (`_greedy_decode`). This is arguably the most complex part of the codebase — it has to interleave normal token-by-token generation with forced tokens for seeds and CDR grafting, while also checking that conserved residues (like the cysteine before CDR1 or the tryptophan after CDR2) land in the right place.

tokenizer.py converts between amino acid letters and the integer tokens the model expects, using `vocab.json`.

utils.py holds the filtering logic — wrappers around ANARCII, Humatch, and AbLang2 — plus various lookup tables used for CDR position checks.

cli.py is a thin wrapper that parses command-line arguments, figures out whether the input is a raw sequence, a FASTA file, or a CSV file, and calls into `pretrained.py` accordingly.

5. Dependencies and Technologies

The core stack is PyTorch for the model and BioPython for sequence handling. The package is built with `hatchling` and distributed via pip.

The three optional tools (ANARCII, Humatch, AbLang2) are each separate packages with their own installation requirements, which is one of the more involved parts of getting the project running — more on this below. ANARCII in particular also brings in HMMER as a system-level dependency when used through Humatch.

6. Setup — What We Did and Where the README Falls Short

The README's installation instructions assume `conda` is already installed and available on the system. On a fresh machine (no conda), the README doesn't mention this step at all, which means a new user following the instructions literally will get stuck at the very first command.

In our case, we did not have conda installed, so before any of the README steps would work we had to install Miniconda first:

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-
x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh
~/miniconda3/bin/conda init bash
```

After this, a new shell session is needed for the `conda` command to be available, and only then can the README's `conda create -n LICHEN_env ...` steps be followed.

A related issue is that the system we used had Python 3.13 as the default Python version, but LICHEN (and especially the optional Humatch/ANARCI path) expects Python 3.9–3.10 due to dependency constraints in `ablang2`, `anarcii`, and `Humatch`. We had to explicitly pin the environment to Python 3.10 when creating it (`conda create -n LICHEN_env python=3.10`), otherwise several of the pip installs failed or installed incompatible versions.

So in practice, the working setup we used was:

```
# Install conda if not already present
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-
x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh
~/miniconda3/bin/conda init bash
# (restart shell)

# Create environment with a compatible Python version
conda create -n LICHEN_env python=3.10
conda activate LICHEN_env

conda install anaconda::pip
conda install -c conda-forge biopython -y
conda install conda-forge::pytorch
conda install conda-forge::anarcii
pip install ablang2
pip install .
```

We initially tried a plain `python -m venv venv` setup instead of conda, since that's what we're more used to for Python projects. This mostly worked for the core LICHEN package and PyTorch, but the optional ANARCII/Humatch installation path in the README specifically assumes conda (it uses `conda install -c bioconda hmmer=3.3.2`), and HMMER isn't readily installable via pip. For anyone who wants the full filtering functionality, conda (or at least a conda-installed HMMER) seems necessary, so we'd recommend the README make this requirement explicit rather than presenting venv and conda as interchangeable.

Once the environment was set up correctly and the model weights were downloaded from Zenodo, the model loaded and ran successfully for both single-sequence and CDR-constrained generation.

7. Improvement Suggestions

Improvement 1: Automatic Model Weight Download

Problem: Right now, anyone setting up LICHEN has to manually go to Zenodo, download the model weights, and put them in the right folder. It's not a huge deal, but it's an unnecessary step that trips people up, especially when the download link isn't prominently visible in the README.

Recommendation: Add a small helper inside the LICHEN class that checks whether the weights file exists before trying to load it. If it doesn't, it should download it automatically from Zenodo using the requests library, or from Hugging Face if the weights get mirrored there. Something like a one-time download on first use, similar to how many other ML libraries handle pretrained weights.

Benefits: New users can go from install to running the model without any manual steps in between. It also means the README setup section becomes shorter and less error-prone.

Improvement 2: FastAPI REST Service

Problem: LICHEN works well as a Python library and CLI tool, but if you want to call it from a web application, a pipeline written in another language, or a microservice, you're stuck wrapping it in a subprocess call — which is messy and fragile.

Recommendation: Build a small FastAPI wrapper around the existing pretrained.py functions. Two endpoints would cover most use cases: POST /generate (accepts a heavy chain plus any optional constraints, returns generated light chains as JSON) and POST /score (accepts a heavy/light pair, returns log-likelihood and perplexity). The existing API maps onto these almost directly, so it wouldn't require changes to any of the core inference code.

Benefits: Makes it straightforward to deploy LICHEN as a proper service. External systems can call it over HTTP regardless of what language they're written in, and you get things like authentication and rate limiting essentially for free by adding standard FastAPI middleware.

Improvement 3: Structured Logging

Problem: Throughout the codebase, feedback and progress information is handled with plain print statements. This works fine for interactive use, but in any kind of production or automated setting it's limiting — you can't easily redirect output, filter by severity, or pipe logs into a monitoring tool.

Recommendation: Swap out the print calls for Python's built-in logging module. Use logging.info for normal progress messages, logging.warning for things like the requested CPU count being higher than what's available, and logging.error before any exception is raised. The verbose flag that already exists in the API can map directly to switching between DEBUG and INFO log levels.

Benefits: Log output becomes controllable without touching the source code. In a deployment setting, logs can be redirected to files or external monitoring systems trivially. It also makes debugging a lot less painful when something goes wrong in a batch job running overnight.

Improvement 4: Add Unit Testing

Problem: There are no automated tests. The CDR grafting logic, tokenizer, model loading, and filtering pipeline all do fairly complex things, and right now the only way to know if a change broke something is to run it manually. That's fine when one person is working on it, but it doesn't scale.

Recommendation: Set up a pytest test suite covering the core pieces: a round-trip encode/decode check on the tokenizer, a model load test on CPU with a small known-good weights file, a basic inference run checking that the output is a valid amino acid string, and at least a few cases for the CDR position-detection functions using known heavy/light pairs with verified CDR positions under both IMGT and Kabat numbering.

Benefits: Regressions get caught automatically instead of by accident. It also makes the codebase much friendlier to new contributors who want to make changes without worrying about breaking something subtle in the inference logic.

Improvement 5: Containerization

Problem: Getting LICHEN fully installed — with the right Python version, conda environment, optional HMMER binary, and model weights in the right place — takes a while and has several places where things can go wrong. We ran into a few of them ourselves during setup.

Recommendation: Provide a Dockerfile that handles the full environment: correct Python version, all conda and pip dependencies, HMMER if needed, and a script that fetches the model weights on first run. A docker-compose.yml could optionally bring up the FastAPI service from Improvement 2 alongside it. The container should default to CPU mode but work with GPU passthrough for people who have CUDA available.

Benefits: Anyone with Docker can have LICHEN running in a few minutes without worrying about conda environments or Python version conflicts. It also makes cloud deployment much simpler — the same container image runs locally and in production.

Improvement 6: Batch GPU Inference

Problem: The decoding loop currently generates one sequence at a time, even when light_generation_bulk is called with a large DataFrame of heavy chains. The GPU ends up mostly idle because it's only ever working on a single sequence worth of computation at a time.

Recommendation: Refactor the decoding loop to support a proper batch dimension, so multiple heavy chains can be encoded and decoded in a single forward pass. PyTorch's Transformer implementation already supports batched inputs — the main work is handling variable-length sequences with padding masks and making sure the per-sequence constraint logic (seeds, CDR grafts) still applies correctly to the right sequence in the batch.

Benefits: For anyone running bulk jobs, throughput could improve by a significant factor. A mid-range GPU that's currently running at maybe 10–15% utilization could get much closer to full capacity with proper batching, which matters a lot when processing hundreds or thousands of antibody sequences.

Optional Implementation

Implemented Improvement: Structured Logging

Out of the six improvements listed, I picked structured logging to actually implement during the assessment window. It felt like the right choice for a few reasons: it doesn't touch any of the inference logic so the risk of breaking something is essentially zero, it's completely self-contained, and honestly it's the kind of change that makes life noticeably easier for anyone who ends up running or debugging this tool in a real setting.

What I did was set up a single logger at the package level in `__init__.py` and updated each module to retrieve it by name rather than calling print. The verbose parameter in light_generation

and light_generation_bulk now controls the log level (DEBUG when verbose=True, INFO otherwise) instead of gating print calls. Edge case warnings — like when the requested CPU count is higher than what's actually available — now go through logging.warning so they show up regardless of the verbosity setting. And any place where the code raises an exception without any prior feedback now logs at logging.error first, so the traceback has a bit more context around it.

8. Documentation

Setup instructions used (working):

```
# 1. Install Miniconda (if conda is not already available)
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-
x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh
~/miniconda3/bin/conda init bash
# restart the shell after this step

# 2. Clone the repository
git clone https://github.com/oxpig/LICHEN.git
cd LICHEN/

# 3. Create environment with a Python version compatible with all
dependencies
conda create -n LICHEN_env python=3.10
conda activate LICHEN_env

# 4. Install dependencies
conda install anaconda::pip
conda install -c conda-forge biopython -y
conda install conda-forge::pytorch
conda install conda-forge::anarcii
```

```
pip install ablang2
pip install .

# 5. Download model weights from Zenodo and place them in the model/
directory
```

Assumptions made during analysis:

- The reviewed codebase corresponds to the version of LICHEN described in the accompanying knowledge-transfer notes, including module names (`pretrained.py`, `inference.py`, `model.py`, etc.) and the described architecture (512-dim embeddings, 6 encoder/6 decoder layers, 8 attention heads).
- "Filtering" behaviour (generating extra candidates and selecting down) is as described in the project documentation and was not independently benchmarked for exact multipliers or timings.

Challenges encountered:

- The default system Python (3.13) was incompatible with the optional ANARCII/Humatch/AbLang2 dependencies, requiring an explicit `python=3.10` environment.
- Conda was not pre-installed, requiring a Miniconda installation step that the README does not mention.
- The README presents `venv` and `conda` as roughly interchangeable, but the optional filtering tools (specifically HMMER via Humatch) effectively require conda.

Future enhancement recommendations:

- Beam search decoding as an alternative to top-p sampling.
- An environment validation script/CLI flag.
- Clearer warnings when CDR grafting cannot be confidently placed.
- Reduced redundant generation when filtering is enabled, via adaptive batch generation.
- A small regression test suite covering CDR position detection for both IMGT and Kabat schemes.
